Property Decorator: Getter and Setter



KEY IDEAS

- `@property` turns a method into a **managed attribute**: callers write `obj.x`, not `obj.x()`. Add validation or computation without changing the public API.
- Getter `@property`, setter `@x.setter`, deleter `@x.deleter`. No setter means **read-only**.
- Store the real value under a different name (`_x`). `self.x = v` in `__init__` goes through the setter, so validation always runs.

TIP Python style: start with a plain public attribute. If you later need validation, switch to `@property`; callers' code does not change. No Java-style getters/setters up front.

EXAMPLE

```
class Temp:
    def __init__(self, celsius):
        self.celsius = celsius        # uses the setter

    @property
    def celsius(self):                # getter
        return self._celsius

    @celsius.setter
    def celsius(self, value):         # setter + validation
        if value < -273.15:
            raise ValueError("below absolute zero")
        self._celsius = value

    @celsius.deleter
    def celsius(self):                # deleter
        del self._celsius

    @property
    def fahrenheit(self):             # read-only, computed
        return self._celsius * 9 / 5 + 32
```

```
t = Temp(25)
t.celsius = 30
print(t.celsius, t.fahrenheit)        # → 30 86.0
# t.fahrenheit = 100  → AttributeError (no setter)
# Temp(-300)          → ValueError: below absolute zero
```